

STENCIL

Technical Specification — Internal Architecture

Version 2.0 · April 2026 · Confidential

1. Overview

This document describes the internal architecture of the Stencil CLI tool. It is intended for engineers building or contributing to Stencil — not for users of the tool. For DSL reference and user-facing features, see the Stencil Product Specification.

Stencil is written in Go. It is a single static binary with no runtime dependencies. It reads a stencil.yaml spec file and produces source code files on disk. The process is entirely deterministic.

Repository structure

stencil/	
cmd/stencil/	← CLI entrypoint (cobra commands)
internal/	
spec/	← SpecAST, ResolvedSpec, parser, validator, resolver
plan/	← GenerationPlan, Task, DAG, topological sort
generator/	← Generator interface, registry, orchestrator
generators/	
go/	← Go-specific generators
table/	← model, repository, errors
api/	← context, hooks, service, handler, dto, mapper
infra/	← externals, messaging, cache, auth, wire
shared/	← migration, types, config, gomod
java/	← Java generators (Phase 2)
imports/	← ImportSet, ImportCollector, hierarchy validator
template/	← Engine, FuncMap, postprocess
emitter/	← File writer, chmod, lock file
diff/	← Lock schema, spec comparison, SpecChange
templates/	
go/	
table/	← model.go.tmpl, repository.go.tmpl, errors.go.tmpl
api/	← context.go.tmpl, hooks.go.tmpl, service.go.tmpl, handler.go.tmpl, dto.go.tmpl, mapper.go.tmpl
infra/	← external.go.tmpl, cache.go.tmpl, producer.go.tmpl
shared/	← migration.sql.tmpl, types.go.tmpl, wire.go.tmpl
shared/partials/	← hook_call.tmpl, flag_check.tmpl, error_wrap.tmpl
testdata/	
golden/	← golden file fixtures per generator
integration/	← full end-to-end test projects
e2e/	← end-to-end tests (compiled binary)
Makefile	
go.mod	

2. Core Data Structures

2.1 SpecAST

Raw output of the parser. Directly mirrors the YAML. No defaults filled, no inference. All enums are still strings.

```
type SpecAST struct {
    Version      int
    Project      string
    Lang         string
    Framework    string
    DB           string
    Config       []ConfigVarAST
    Tables       []TableAST
    Types        []CustomTypeAST
    Transactions []TransactionAST
    Externals    []ExternalAST
    Messaging    *MessagingAST
    Cache        *CacheAST
    Storage      *StorageAST
    Auth         *AuthAST
    Observability *ObservabilityAST
    Middleware   []string
    Resources    []ResourceGroupAST // API groups
    Extensions   []ExtensionAST
    Overrides    *OverridesAST
}

type ResourceGroupAST struct {
    Group      string
    BasePath   string
    Auth       string
    APIs       []APIAST
}

type APIAST struct {
    Name      string
    Method     string
    Path       string
    Auth       string
    Roles      []string
    Owner      bool
    Request    string           // DTO name reference or inline
    Response   string
    DTOs       *DTOblockAST      // inline DTO definitions
    Touches    []TouchAST        // ordered infra interactions
}

type TouchAST struct {
    // exactly one of these is set:
    Table      string // table name
    External   string // external name
    Message     string // event name
    Cache       string // cache interface name
    Transaction string // transaction name
    Storage     string // storage op name
}
```

```

    Op      string    // create|get|list|update|delete|set|invalidate
    Method  string    // for external touches
    Flag    string    // control flag name in shared context
    Default bool      // initial value of control flag
}

```

2.2 ResolvedSpec

Output of the Resolver. Every implicit value is explicit. All types resolved. All indexes derived. All shared context structs planned. This is what generators receive.

```

type ResolvedSpec struct {
    Project      string
    Module       string          // Go module path
    Lang         Lang
    Framework    Framework
    DB           DBDriver
    Config       []ResolvedConfigVar
    Tables       []ResolvedTable
    Types        TypeRegistry    // map[string]ResolvedCustomType
    Transactions []ResolvedTransaction
    Externals    []ResolvedExternal
    Messaging    *ResolvedMessaging
    Cache        *ResolvedCache
    Auth         *ResolvedAuth
    Resources    []ResolvedResourceGroup
    Overrides    ResolvedOverrides
}

type ResolvedResourceGroup struct {
    Group      string
    BasePath   string
    Auth       ResolvedAuth
    APIs       []ResolvedAPI
}

type ResolvedAPI struct {
    Name          string
    Method         string
    FullPath       string          // base_path + path, resolved
    Auth          ResolvedAuth
    RequestDTO     *ResolvedDTO
    ResponseDTO    *ResolvedDTO
    Touches        []ResolvedTouch // ordered
    SharedContext  ResolvedSharedContext
    HookInterface  ResolvedHookInterface
    Mappers        []ResolvedMapper
}

type ResolvedTouch struct {
    Kind      TouchKind // Table|External|Message|Cache|Transaction|Storage
    Name      string    // resolved name of the infra item
    Op        string
    Flag      string    // control flag field name
    Default   bool
    // resolved references to actual infra items:
    TableRef  *ResolvedTable
}

```

```

    ExternalRef      *ResolvedExternal
    TransactionRef   *ResolvedTransaction
    CacheRef         *ResolvedCacheInterface
}

type ResolvedSharedContext struct {
    TypeName string      // e.g. "CreateUserContext"
    Fields    []ContextField // all fields: request + flags + results + errors +
response
}

type ContextField struct {
    Name      string
    GoType     string
    Kind      ContextFieldKind // Request|Flag|Result|Error|Response
    Default   interface{}          // for flags: true or false
}

type ResolvedHookInterface struct {
    TypeName string
    Hooks      []ResolvedHook
}

type ResolvedHook struct {
    Name      string // e.g. "BeforeCreateUser", "AfterTableUsersCreate"
    Signature string // full Go function signature
    When      HookWhen // Before|After
    TouchIdx  int      // which touch this hook is for (-1 = entry/exit)
}

```

2.3 GenerationPlan, Task, File

```

type GenerationPlan struct {
    Tasks []Task
    Tiers [][]Task // topologically sorted for parallel execution
    Reason PlanReason // FirstRun | Update | AddTable | AddAPI
}

type Task struct {
    ID          string
    GeneratorID string
    Context     GeneratorContext
    DependsOn   []string
}

type GeneratorContext struct {
    Spec      *ResolvedSpec
    Table     *ResolvedTable // nil for API and cross-cutting generators
    API       *ResolvedAPI    // nil for table and cross-cutting generators
    Group     *ResolvedResourceGroup
    Changes   []SpecChange
    OutputDir string
}

type File struct {
    Path      string
    Content   []byte
    ReadOnly  bool
}

```

}

3. Pipeline Stages

3.1 Parser

Reads stencil.yaml bytes. Two sub-steps: yaml.v3 produces generic map, GoForge parser maps to typed SpecAST. Parser never validates — that is the Validator's job.

```
func (p *Parser) Parse(data []byte) (*SpecAST, error) {
    var raw map[string]interface{}
    if err := yaml.Unmarshal(data, &raw); err != nil {
        return nil, &ParseError{Line: extractLine(err), Msg: err.Error()}
    }
    ast := &SpecAST{}
    return ast, mapToAST(raw, ast)
}
```

3.2 Validator

All semantic checks in one pass. Never stops at first error. Returns []ValidationError with line numbers and machine-readable codes.

Category	Checks
Top-level	version, project, lang, framework, db, config, resources all present
Tables	Field types resolve to primitive or declared type. Enum has values list. State machine field exists and is enum. FK referenced in relations exists.
Types	No circular references. No type references a resource.
Externals	base_url references a declared config var. All call methods have valid HTTP verbs.
Resources	Every API has at least one touch. Touch references resolve — table names, external names, message names, cache names, transaction names all exist.
DTOs	Fields on inline DTOs must be declared types or primitives. private: true table fields cannot appear in any response DTO.
Touch flags	No two touches in the same API share a flag name. Flag names are valid Go identifiers.
Cache	value_type in cache interface resolves to a known DTO or type.
Auth	Roles on endpoints exist in auth.roles list. owner: true only on APIs touching a table with a user_id field.

3.3 Resolver

Takes SpecAST, produces ResolvedSpec. Fills all defaults, derives all implicit values, plans all shared context structs and hook interfaces.

Resolver steps — in order

Step	Name	What it does
1	Build TypeRegistry	Index all custom types by name for O(1) lookup during field resolution.
2	Resolve lang/db/fw	Convert string enums to typed Go constants (LangGo, DBPostgres, FrameworkGin).
3	Resolve tables	Fill GoType/JavaType/DBType for each field. Derive indexes from queries. Wire relations. Expand soft_delete and timestamps.
4	Resolve externals	Fill typed request/response structs. Derive error mappings. Plan mock interface.
5	Resolve cache	Fill typed Get/Set/Delete/Invalidate signatures from value_type.
6	Resolve transactions	Plan typed params structs and result types for each step.
7	Resolve touches	For each touch: resolve reference to infra item. Fill flag name (default Run{Kind}{Name}{Op}). Confirm default.
8	Plan shared context	For each API: build ResolvedSharedContext — all flag fields, all result fields, all error fields, request field, response field.
9	Plan hook interface	For each API: derive all hook points. Entry (BeforeAPI), one Before+After per touch, BeforeResponse. Add ComputeXxx hooks for compute: true DTO fields.
10	Resolve DTOs	Inline DTOs expanded. External DTO references validated. Mappers planned (which table model fields map to which DTO fields).
11	Derive DI graph	For each API service: determine constructor params (which repos, clients, producers, caches are needed based on touches).
12	Validate import DAG	Check no circular imports. Error if cycle detected.

3.4 Diff Planner and DAG

Compares ResolvedSpec against lock file. Produces GenerationPlan. Builds a DAG from generator dependencies. Runs Kahn's algorithm for topological sort into parallel execution tiers.

Generator dependency map

```
var generatorDeps = map[string][]string{
    // tier 1 — no dependencies
    "sql.migration": {},
    "go.types":      {},
    "go.errors":     {},
    "go.config":     {},

    // tier 2 — depend on types
    "go.table.model": {"go.types", "go.errors"},
    "go.api.dto":     {"go.types", "go.errors"},
    "go.cache":       {"go.types"},

    // tier 3 — depend on model and dto
    "go.table.repo": {"go.table.model"},
    "go.api.mapper": {"go.table.model", "go.api.dto"},
    "go.api.context": {"go.api.dto"},
    "go.api.hooks":  {"go.api.context"},

    // tier 4 — depend on repo, mapper, context, hooks
    "go.api.service": {"go.table.repo", "go.api.mapper", "go.api.hooks",
"go.cache", "go.external", "go.tx"},
    "go.external":    {"go.errors"},
    "go.messaging":   {"go.types"},
    "go.tx":          {"go.table.repo"},
    "go.auth":        {"go.table.repo"},

    // tier 5 — depend on service
    "go.api.handler": {"go.api.service", "go.api.dto"},
    "go.routes":      {"go.api.handler", "go.auth"},

    // tier 6 — depend on everything
    "go.wire":        {"go.api.service", "go.auth", "go.external", "go.messaging",
"go.cache"},
    "go.mod":         {"go.wire"},
}
```

Kahn's algorithm — TopologicalTiers

```
func (d *DAG) TopologicalTiers(tasksByID map[string]Task) ([][]Task, error) {
    queue := []string{}
    for node := range d.nodes {
        if d.inDegree[node] == 0 {
            queue = append(queue, node)
        }
    }
    sort.Strings(queue) // deterministic within tier

    tiers := [][]Task{}
    processed := 0
    for len(queue) > 0 {
        tier := []Task{}
        for _, nodeID := range queue {
            tier = append(tier, tasksByID[nodeID])
            processed++
        }
        tiers = append(tiers, tier)
        nextQueue := []string{}
        for _, nodeID := range queue {
            for _, dep := range d.edges[nodeID] {

```

```
        d.inDegree[dep]--
        if d.inDegree[dep] == 0 {
            nextQueue = append(nextQueue, dep)
        }
    }
    sort.Strings(nextQueue)
    queue = nextQueue
}
if processed < len(d.nodes) {
    return nil, fmt.Errorf("dependency cycle: %s", findCycle(d))
}
return tiers, nil
}
```

4. Generator System

4.1 Generator Interface

```
type Generator interface {
    ID() string
    Generate(ctx GeneratorContext) ([]File, error)
}

// Generators are pure functions. No filesystem I/O. No global state.
// Return []File. Emitter writes to disk.
```

4.2 API Generator Suite

The most important generators. For each API, six generators run in dependency order to produce the full set of generated files.

Generator	Generates	Key inputs from ResolvedAPI
go.api.dto	dto.go — request + response DTO structs	RequestDTO, ResponseDTO, compute fields
go.api.context	context.go — shared context struct	SharedContext.Fields — all flags, results, errors, request, response
go.api.hooks	hooks.go — hook interface with all hook points	HookInterface.Hooks — all hook names and signatures
go.api.mapper	mapper.go — DTO ↔ table model conversions	Mappers — which model fields map to which DTO fields
go.api.service	service.go — executor: flag reads, infra calls, hook callsites	Touches (ordered), SharedContext, HookInterface
go.api.handler	handler.go — HTTP parse/respond layer	Method, FullPath, RequestDTO, ResponseDTO, Auth

4.3 Service Generator — The Core of the System

The service generator is the most complex. It reads the ordered Touches list and for each touch renders: the flag check, the infra call, and the hook callsites. The template never contains conditional logic — all decisions are in the template data.

Template data struct

```
type ServiceTemplateData struct {
```

```

    PackageName    string
    Imports        imports.ImportSet
    APIName        string
    ContextType    string           // "CreateUserContext"
    HooksType      string           // "CreateUserHooks"
    ValidatorCall  string           // generated validation call
    Steps          []ServiceStep    // one per touch, in declaration order
    FallbackMapper string          // mapper call if Response still nil
}

type ServiceStep struct {
    Flag        string           // "RunTableUsersCreate"
    InfraCall   string           // pre-rendered infra call code
    ResultField string           // "TableUsersResult"
    ErrorField  string           // "TableUsersError"
    BeforeHook  *HookCallSite                // nil if no before hook
    AfterHook   *HookCallSite                // nil if no after hook
    FatalError  bool              // true: return on error; false: store in
    context
}

type HookCallSite struct {
    FieldName string // field on hooks struct e.g. "AfterTableUsersCreate"
    ArgsCode  string // pre-rendered argument list
}

```

Service template — reads flags, never decides

```

// templates/go/api/service.go.tmpl

func (s *({{ .APIName }}Service) ({{ .APIName }}(ctx context.Context, req ({{
    .RequestType }}) ({{ .ResponseType }}, error) {

    shared := &({{ .ContextType }}{
        Request: req,
        {{ range .Steps }}
        {{ .Flag }}: {{ if .DefaultTrue }}true{{ else }}false{{ end }},
        {{ end }}
    )

    ({{ .ValidatorCall }})

    if s.hooks.Before({{ .APIName }}) != nil {
        if err := s.hooks.Before({{ .APIName }}(ctx, shared); err != nil {
            return nil, err
        }
    }

    {{ range .Steps }}
    if shared.{{ .Flag }} {
        ({{ .InfraCall }})
        {{ if .BeforeHook }}
        if s.hooks.{{ .BeforeHook.FieldName }} != nil {
            if err := s.hooks.{{ .BeforeHook.FieldName }}(ctx, shared); err != nil
        {
            return nil, err
        }
        {{ end }}
    }
    {{ end }}
    {{ if .FatalError }}

```

```

        if shared.{{ .ErrorField }} != nil {
            return nil, shared.{{ .ErrorField }}
        }
        {{ end }}
        {{ if .AfterHook }}
        if s.hooks.{{ .AfterHook.FieldName }} != nil {
            if err := s.hooks.{{ .AfterHook.FieldName }}(ctx, shared); err != nil {
                return nil, err
            }
        }
        {{ end }}
    }
    {{ end }}

    if s.hooks.BeforeResponse != nil {
        if err := s.hooks.BeforeResponse(ctx, shared); err != nil {
            return nil, err
        }
    }
    if shared.Response == nil {
        {{ .FallbackMapper }}
    }
    return shared.Response, nil
}

```

Template purity

The service template contains zero if/else logic of its own. All branching is in the template data — Steps is pre-ordered, FatalError is pre-determined, InfraCall is pre-rendered. The template is a mechanical substitution layer only. This keeps templates testable and maintainable.

4.4 Context Generator

Generates the shared context struct. Reads ResolvedSharedContext.Fields and renders them in four groups: request, flags, results+errors, response.

```

// templates/go/api/context.go.tmpl

type {{ .TypeName }} struct {

    // — Request —————
    Request {{ .RequestType }}

    // — Control flags —————
    {{ range .FlagFields }}
    {{ .Name }} bool    // default: {{ if .Default }}true{{ else }}false{{ end }}
    {{ end }}

    // — Infra results —————
    {{ range .ResultFields }}
    {{ .Name }} {{ .GoType }}
    {{ end }}

    // — Infra errors —————
    {{ range .ErrorFields }}

```

```

    {{ .Name }} error
    {{ end }}

    // — Response —————
    Response {{ .ResponseType }}
}

```

4.5 Hooks Generator

Generates the hook interface. Every hook receives the full shared context — giving access to all previous results and the ability to set any future flag.

```

// templates/go/api/hooks.go.tmpl

type {{ .TypeName }} struct {
    // Entry — inspect request, set initial flags, mutate request
    Before{{ .APIName }} func(ctx context.Context, shared *{{ .ContextType }})
    error

    {{ range .TouchHooks }}
    {{ if .HasBefore }}
    // Before {{ .TouchDescription }}
    {{ .BeforeName }} func(ctx context.Context, shared *{{ $.ContextType }}) error
    {{ end }}
    {{ if .HasAfter }}
    // After {{ .TouchDescription }}
    {{ .AfterName }} func(ctx context.Context, shared *{{ $.ContextType }}) error
    {{ end }}
    {{ end }}

    // Build final response — set shared.Response
    BeforeResponse func(ctx context.Context, shared *{{ .ContextType }}) error

    {{ range .ComputeHooks }}
    // Computed DTO field
    {{ .Name }} func(ctx context.Context, shared *{{ $.ContextType }}) error
    {{ end }}
}

```

5. Import Management

5.1 Import Collector — per generator

Each generator has a dedicated collector function. The collector walks the ResolvedAPI or ResolvedTable and builds the exact ImportSet. No generator infers imports at template time.

```
func APIServiceImports(api spec.ResolvedAPI, mod string) ImportSet {
    s := NewImportSet(mod)
    s.Add("context")
    s.Add("fmt")

    for _, touch := range api.Touches {
        switch touch.Kind {
        case spec.TouchKindTable:
            s.Add(mod + "/generated/tables/" + touch.Name)
        case spec.TouchKindExternal:
            s.Add(mod + "/generated/externals")
        case spec.TouchKindMessage:
            s.Add(mod + "/generated/messaging")
        case spec.TouchKindCache:
            s.Add(mod + "/generated/cache")
        case spec.TouchKindTransaction:
            s.Add(mod + "/generated/transactions")
        }
    }
    for _, field := range api.RequestDTO.Fields {
        if field.Type.Kind == spec.TypeUUID { s.Add("github.com/google/uuid") }
        if field.Type.Kind == spec.TypeTimestamp { s.Add("time") }
        if field.Type.Kind == spec.TypeCustom { s.Add(mod + "/generated/types") }
    }
    return s
}
```

5.2 Import hierarchy — circular dependency prevention

Layer	May only import from
generated/types/	Standard library only
generated/tables/{t}/model.go	types/ only
generated/tables/{t}/repository.go	model.go
generated/cache/	types/ only
generated/externals/	types/ only
generated/apis/{api}/dto.go	types/ only

<code>generated/apis/{api}/context.go</code>	<code>dto.go, types/</code>
<code>generated/apis/{api}/hooks.go</code>	<code>context.go</code>
<code>generated/apis/{api}/mapper.go</code>	<code>model.go, dto.go</code>
<code>generated/apis/{api}/service.go</code>	<code>repo, mapper, hooks, context, cache, externals, messaging, transactions</code>
<code>generated/apis/{api}/handler.go</code>	<code>service.go, dto.go only</code>
<code>generated/wire.go</code>	all services — always generated last

5.3 Post-render processing

Pass	What it does
goimports	Adds missing imports, removes unused, groups stdlib/internal/external. If goimports fails the file has a syntax error — generator bug.
gofmt	Canonical Go formatting. Template whitespace irrelevant — gofmt fixes it.

6. Orchestrator and Emitter

6.1 Orchestrator — parallel tier execution

```
func (o *Orchestrator) Run(plan *plan.GenerationPlan) error {
    for _, tier := range plan.Tiers {
        var wg sync.WaitGroup
        errCh := make(chan error, len(tier))

        for _, task := range tier {
            wg.Add(1)
            go func(t plan.Task) {
                defer wg.Done()
                gen, ok := o.registry.Get(t.GeneratorID)
                if !ok {
                    errCh <- fmt.Errorf("unknown generator: %s", t.GeneratorID)
                    return
                }
                files, err := gen.Generate(t.Context)
                if err != nil {
                    errCh <- fmt.Errorf("%s: %w", t.GeneratorID, err)
                    return
                }
                for _, f := range files {
                    if err := o.emitter.Stage(f); err != nil { errCh <- err }
                }
            }(task)
        }
        wg.Wait()
        close(errCh)

        var tierErrs []error
        for err := range errCh { tierErrs = append(tierErrs, err) }
        if len(tierErrs) > 0 { return errors.Join(tierErrs...) }
    }
    return o.emitter.Flush()
}
```

6.2 Emitter — staged writes with rollback

Files are staged in memory during generation. Only Flush() writes to disk. If any write fails, rollback restores from backup. generated/ is never left in a partial state.

```
func (e *Emitter) Flush() error {
    e.unlockOutputDir() // chmod 755 on generated/
    if err := e.backup(); err != nil { return err }

    for _, f := range e.staged {
        if err := e.writeFile(f); err != nil {
            e.rollback() // restore from backup
            return err
        }
    }

    e.lockOutputDir() // chmod 444 on all files in generated/
}
```

```
    return e.writeLockFile()      // update .stencil.lock  
}
```

7. Folder Structure

```

stencil/
├── cmd/stencil/
│   ├── main.go           ← cobra root, RegisterAll()
│   ├── generate.go
│   ├── update.go
│   ├── diff.go
│   ├── validate.go
│   └── hooks.go          ← "stencil hooks scaffold <api>"
├── internal/
│   ├── spec/
│   │   ├── ast.go        ← SpecAST + all sub-types
│   │   ├── resolved.go   ← ResolvedSpec + all sub-types
│   │   ├── parser.go
│   │   ├── validator.go
│   │   ├── resolver.go
│   │   └── typemap.go    ← DSL type → Go/Java/DB mappings
│   ├── diff/
│   │   ├── lock.go       ← LockFile schema, read/write
│   │   ├── changes.go    ← SpecChange types
│   │   ├── planner.go    ← Plan, partialPlan, buildPlan
│   │   └── dag.go        ← buildDAG, TopologicalTiers
│   ├── generator/
│   │   ├── generator.go  ← Generator interface, File, GeneratorContext
│   │   ├── registry.go   ← Registry, Register, AffectedBy
│   │   └── orchestrator.go
│   ├── generators/
│   │   ├── go/
│   │   │   ├── table/
│   │   │   │   ├── model.go    ← go.table.model
│   │   │   │   ├── repo.go     ← go.table.repo
│   │   │   │   └── errors.go    ← go.errors
│   │   │   └── api/
│   │   │       ├── dto.go       ← go.api.dto
│   │   │       ├── context.go   ← go.api.context
│   │   │       ├── hooks.go     ← go.api.hooks
│   │   │       ├── mapper.go    ← go.api.mapper
│   │   │       ├── service.go   ← go.api.service
│   │   │       └── handler.go   ← go.api.handler
│   │   └── infra/
│   │       ├── external.go      ← go.external
│   │       ├── cache.go        ← go.cache
│   │       ├── messaging.go     ← go.messaging
│   │       ├── auth.go         ← go.auth
│   │       ├── tx.go           ← go.tx
│   │       ├── wire.go         ← go.wire
│   │       ├── routes.go       ← go.routes
│   │       ├── config.go       ← go.config
│   │       └── gomod.go        ← go.mod
│   └── shared/
│       ├── migration.go        ← sql.migration
│       └── types.go            ← shared.types
├── imports/
│   ├── set.go
│   ├── collectors.go          ← per-generator collector functions
│   └── hierarchy.go           ← DAG validation

```

```

├── template/
│   ├── engine.go
│   ├── funcmap.go
│   └── postprocess.go    ← goimports + gofmt
├── emitter/
│   ├── emitter.go
│   └── lock.go
├── templates/
│   ├── go/
│   │   ├── table/
│   │   │   ├── model.go.tmpl
│   │   │   ├── repository.go.tmpl
│   │   │   └── errors.go.tmpl
│   │   ├── api/
│   │   │   ├── context.go.tmpl
│   │   │   ├── hooks.go.tmpl
│   │   │   ├── service.go.tmpl
│   │   │   ├── handler.go.tmpl
│   │   │   ├── dto.go.tmpl
│   │   │   └── mapper.go.tmpl
│   │   └── infra/
│   │       ├── external.go.tmpl
│   │       ├── external_mock.go.tmpl
│   │       ├── cache.go.tmpl
│   │       ├── producer.go.tmpl
│   │       ├── consumer.go.tmpl
│   │       ├── auth.go.tmpl
│   │       ├── tx.go.tmpl
│   │       ├── wire.go.tmpl
│   │       ├── routes.go.tmpl
│   │       └── config.go.tmpl
│   ├── sql/
│   │   └── migration.sql.tmpl
│   └── shared/partials/
│       ├── flag_check.tmpl    ← "if shared.RunXxx {" pattern
│       ├── hook_call.tmpl    ← nil check + call pattern
│       └── error_wrap.tmpl
├── testdata/
│   ├── golden/
│   │   ├── go_api_context/
│   │   │   ├── createuser_basic/
│   │   │   │   ├── spec.yaml
│   │   │   │   └── context.go.golden
│   │   │   ├── createuser_with_conditional_external/
│   │   │   ├── getuser_with_cache_readonly/
│   │   │   └── checkout_multi_touch/
│   │   ├── go_api_hooks/
│   │   ├── go_api_service/
│   │   ├── go_api_handler/
│   │   ├── go_api_dto/
│   │   ├── go_table_model/
│   │   ├── go_table_repo/
│   │   ├── go_external/
│   │   ├── go_cache/
│   │   └── sql_migration/
│   └── integration/
│       ├── orders_service/
│       │   └── stencil.yaml
│       └── expected/    ← full expected output tree

```

```
└─ minimal_service/
└─ e2e/
    ├── generate_test.go
    ├── update_test.go
    ├── compile_test.go
    └── runtime_test.go
└─ Makefile
└─ go.mod
└─ README.md
```

8. Testing Strategy

The core question per layer

Layer 1 (unit): does the pipeline produce correct internal data structures? Layer 2 (golden): does each generator produce the expected source text? Layer 3 (compile): does the assembled output compile? Layer 4 (runtime): does the generated service behave correctly at HTTP level? Layer 5 (regression): does a spec change produce exactly the expected file diff?

8.1 Layer 1 — Unit tests

Table-driven tests for Parser, Validator, Resolver, DAG. No filesystem. The Resolver is the most important to unit test — specifically that SharedContext and HookInterface are built correctly from touch declarations.

```
// Key Resolver test: shared context built correctly from touches
func TestResolver_SharedContextFromTouches(t *testing.T) {
    cases := []struct{
        name      string
        touches []TouchAST
        wantFlags []ContextField // flag fields expected
        wantResults []ContextField // result fields expected
    }{
        {
            name: "table touch adds flag + result + error",
            touches: []TouchAST{{Table: "users", Op: "create", Default: true}},
            wantFlags: []ContextField{{Name: "RunTableUsersCreate",
Default: true}},
            wantResults: []ContextField{{Name: "TableUsersResult", GoType: "*User"}},
        },
        {
            name: "external touch with default false adds conditional flag",
            touches: []TouchAST{{External: "CRMClient", Method: "CreateContact",
Default: false}},
            wantFlags: []ContextField{{Name: "RunCRMClientCreate", Default: false}},
        },
    }
    // ...
}
```

8.2 Layer 2 — Golden file tests

One test per generator, multiple scenarios. The -update flag regenerates golden files. The git diff is the code review.

Key golden scenarios for the API generators

Scenario	What it tests
----------	---------------

createuser_basic	Table touch only. Flag default true. BeforeCreateUser + AfterTableUsersCreate + BeforeResponse hooks.
createuser_conditional_external	Table + external with default:false. AfterTableUsersCreate sets flag. AfterCRMClientCreate handles error.
getuser_cache_readthrough	Cache read (default:true) + table (default:true) + cache write. BeforeGetUser sets RunTableUsersGet=false on hit.
chargecard_no_table	External only. No table touch at all. Verifies API works with zero DB involvement.
checkout_multi_touch	Transaction + external + message. Three flag fields. Multiple Before/After hooks.
listusers_with_pagination	Table list touch. Paginated response DTO. No flags beyond RunTableUsersList.

8.3 Layer 3 — Compile test

Generates from integration specs then runs `go build ./...` and `go vet ./...` on the output. Catches type mismatches between generated files that golden tests cannot see.

```
func TestGeneratedCodeCompiles(t *testing.T) {
    if testing.Short() { t.Skip() }
    cases := []string{
        "testdata/integration/minimal_service/stencil.yaml",
        "testdata/integration/orders_service/stencil.yaml",
    }
    for _, spec := range cases {
        t.Run(spec, func(t *testing.T) {
            outDir := t.TempDir()
            require.NoError(t, stencil.Generate(spec, outDir))
            cmd := exec.Command("go", "build", "./...")
            cmd.Dir = outDir
            out, err := cmd.CombinedOutput()
            require.NoError(t, err, "compile failed:\n%s", out)
        })
    }
}
```

8.4 Layer 4 — Runtime test

Generates `orders_service`, runs against real Postgres, hits HTTP endpoints. The only layer that catches private field leakage, auth failures, cache correctness, and flag-driven branching at runtime.

Key runtime assertions

Assertion	Why runtime only
password_hash absent from response	No other layer sends real HTTP and inspects the JSON body.
Cache hit skips DB	Verify via DB query count — only runtime test can count actual queries.
Conditional CRM not called if personal account	Mock CRM records call count — only verifiable at runtime with real hook impl.
State machine rejects invalid transition	Only runtime test can attempt HTTP PATCH and assert 422.
Non-fatal CRM error does not abort create	Only runtime test can inject CRM failure and verify user was still created.
Auth 401 on missing token	Only runtime test sends requests without Authorization header.

8.5 Layer 5 — Update regression test

Verifies that stencil update after a spec change modifies exactly the right files. Both expectChanged and expectUnchanged lists are asserted. Developer files are always in expectUnchanged.

```

cases := []struct {
    name          string
    base, updated  string
    expectChanged  []string
    expectUnchanged []string
}{
    {
        name:      "add conditional external touch to CreateUser",
        base:      "testdata/update/add_touch/before.yaml",
        updated:   "testdata/update/add_touch/after.yaml",
        expectChanged: []string{
            "generated/apis/createuser/context.go",
            "generated/apis/createuser/hooks.go",
            "generated/apis/createuser/service.go",
            "generated/wire.go",
        },
        expectUnchanged: []string{
            "generated/tables/users/model.go",
            "generated/apis/createuser/dto.go",
            "hooks/user/createuser.hooks.go",
        },
    },
}

```

8.6 Makefile targets


```
make test          # unit + golden (fast, runs every commit)
make lint          # golangci-lint on Stencil tool itself
make test-compile  # generate + go build + go vet (runs on PR merge)
make test-update   # update regression tests
make test-idempotency
make test-runtime  # full HTTP runtime tests (runs nightly)
make golden-update # regenerate all golden files – review diff before commit
make dev-generate  # generate from ./stencil.yaml into ./dev-output/
make test-all      # all layers in sequence (release)
make build         # cross-compile for Linux/macOS/Windows amd64+arm64
```

9. Adding a New Generator

Step 1 — Implement the Generator interface

```
type MyGenerator struct { engine *template.Engine }
func (g *MyGenerator) ID() string { return "go.my" }
func (g *MyGenerator) Generate(ctx generator.GeneratorContext) ([]generator.File,
error) {
    data := g.buildData(ctx)
    rendered, err := g.engine.Render("go/my/my.go.tmpl", data)
    if err != nil { return nil, err }
    processed, err := template.PostProcessGo("my.go", rendered)
    if err != nil { return nil, err }
    return []generator.File{{
        Path:      fmt.Sprintf("generated/my/%s/my.go", ctx.API.Name),
        Content:   processed,
        ReadOnly:  true,
    }}, nil
}
```

Step 2 — Write the template

```
// templates/go/my/my.go.tmpl
package {{ .PackageName }}

import (
    {{ renderImports .Imports }}
)

// template body
```

Step 3 — Write the ImportCollector function

```
// internal/imports/collectors.go
func MyImports(api spec.ResolvedAPI, mod string) ImportSet {
    s := NewImportSet(mod)
    // add based on what my generator uses
    return s
}
```

Step 4 — Register in dependency map and RegisterAll

```
// internal/plan/dependencies.go
"go.my": {"go.api.context"}, // add to correct tier

// cmd/stencil/main.go
r.Register(&MyGenerator{engine: engine}, []diff.ChangeKind{
    diff.AddAPI, diff.AddTouch,
})
```

Step 5 — Write golden file tests

```
// testdata/golden/go_my/basic/spec.yaml      ← create
// testdata/golden/go_my/basic/my.go.golden   ← generate with -update, review diff

make golden-update
git diff testdata/golden/go_my/
```

Step 6 — Run full test suite

```
make test      # must pass
make test-compile # generated code must compile
make lint      # zero warnings
```